

Базы данных

Тема 8

Представления. Индексы

Представления

- представление (*view*) - это виртуальная таблица, состоящая из совокупности именованных столбцов и строк данных, содержимое которой определяется запросом:
 - определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
 - пока представление не будет материализовано / проиндексировано, оно не существует в базе данных как хранимая совокупность значений: строки и столбцы динамически формируются при обращении к представлению на основе данных из таблиц (и других представлений), указанных в определяющем представлении запросе – т.е., фактически представление является сохранённым именованным запросом.

Функции представлений

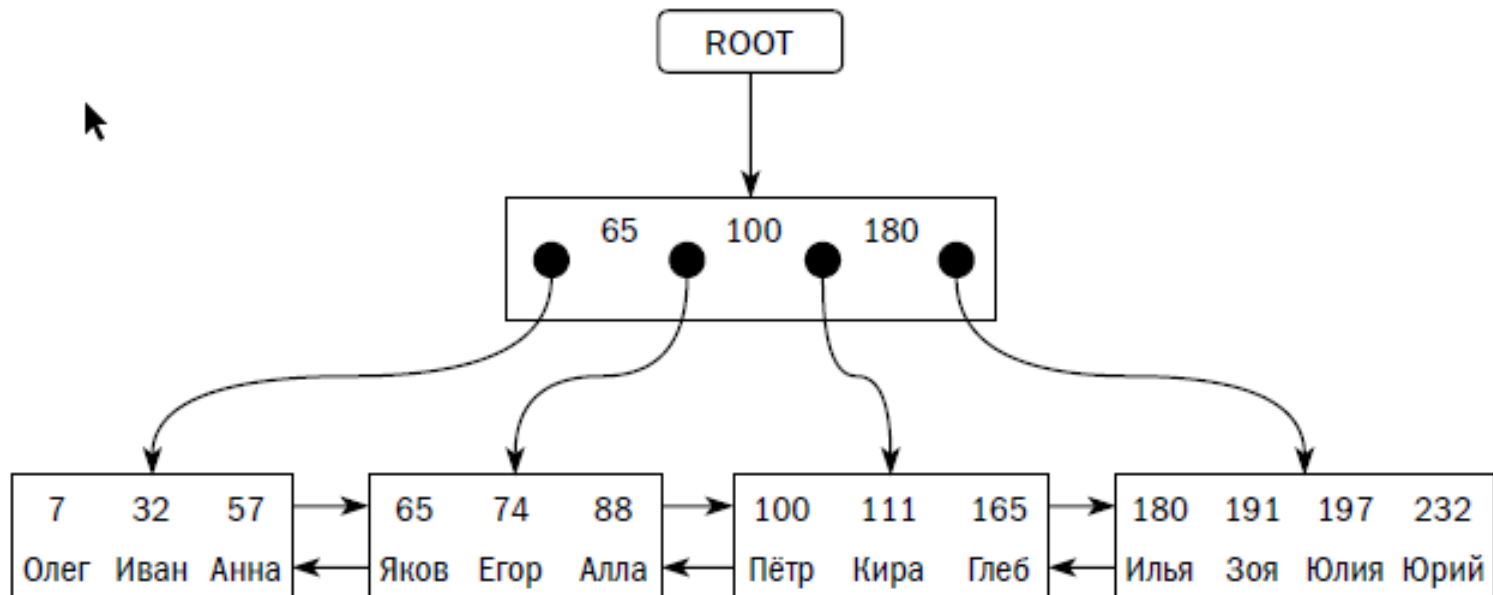
- упрощение и настройка восприятия каждым пользователем информации базы данных;
- реализация механизмов безопасности, обеспечивающих возможность обращения пользователей к данным без предоставления им разрешений на непосредственный доступ к базовым таблицам;
- обеспечение интерфейса обратной совместимости, моделирующего таблицу, которая существует, но схема которой изменилась;
- определение представлений с данными из нескольких гетерогенных источников.

Индексы

- индекс является структурой на диске, которая связана с таблицей или представлением и может способствовать сокращению времени выполнения запроса за счет сокращения количества дисковых операций ввода-вывода и уменьшения потребления системных ресурсов;
- индекс содержит ключи, построенные на основе одного или нескольких столбцов в таблице или представлении;
- индексы могут быть уникальными (т.е. никакие две строки не имеют одинаковое значение для ключа индекса) или неуникальными;
- уникальные индексы создаются автоматически при определении ограничений PRIMARY KEY или UNIQUE на основе столбцов таблицы;
- основной индексной структурой, используемой для организации индексов, является B-дерево.

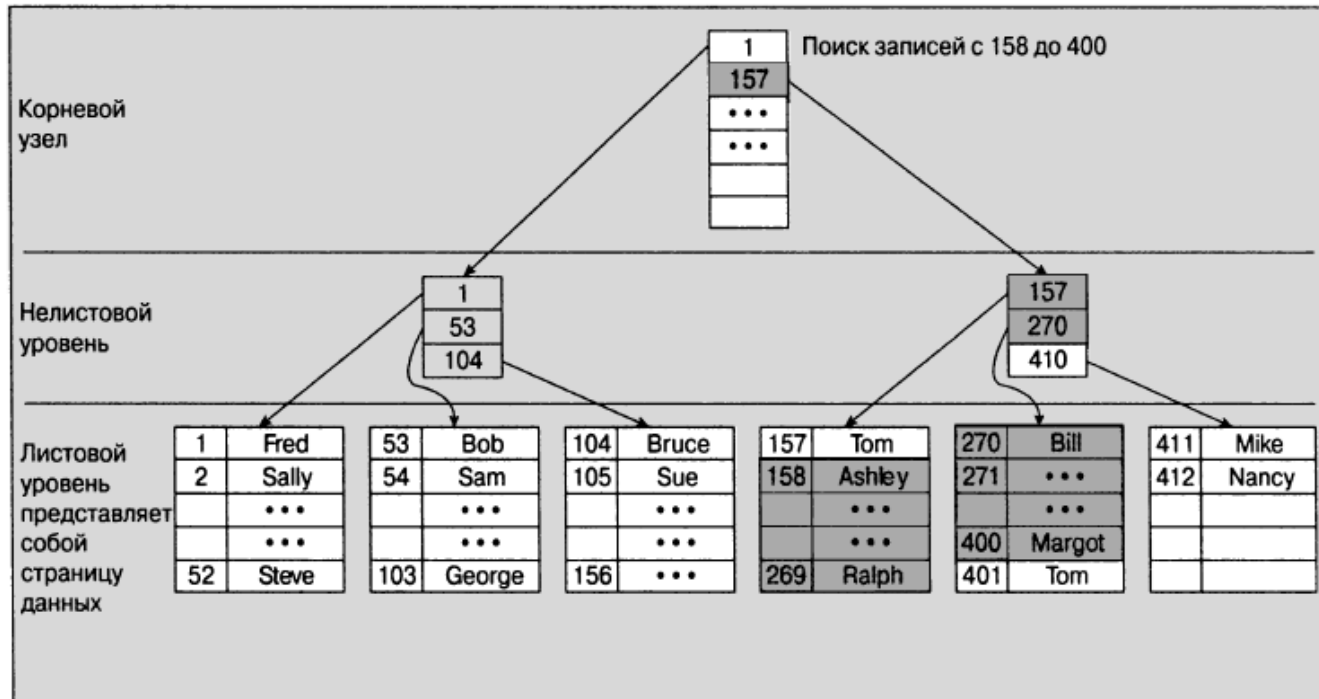
В-дерево

- преимущества:
 - эффективная вставка (количество изменяемых блоков не превышает глубины дерева);
 - эффективный поиск (глубина дерева – $\log N / \log f$);
 - поддерживает поиск по значению (равенство) и по диапазону (неравенства);
 - результаты последовательного чтения индекса упорядочены по значению ключа индекса.



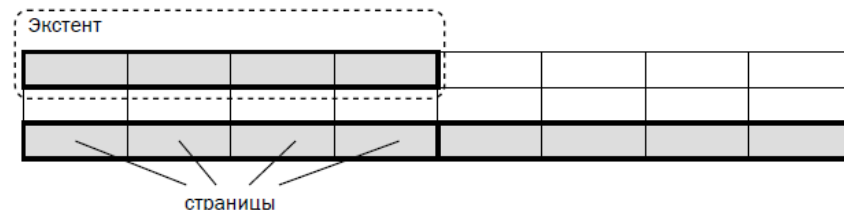
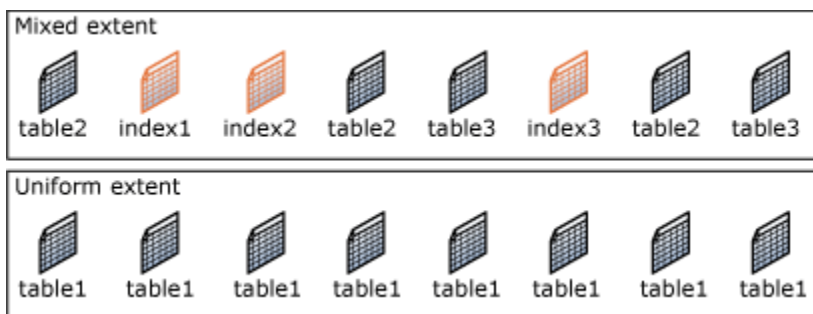
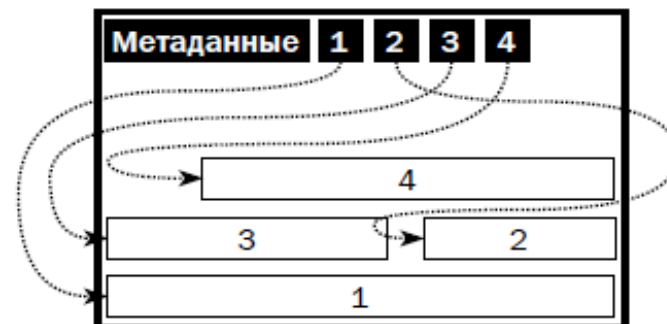
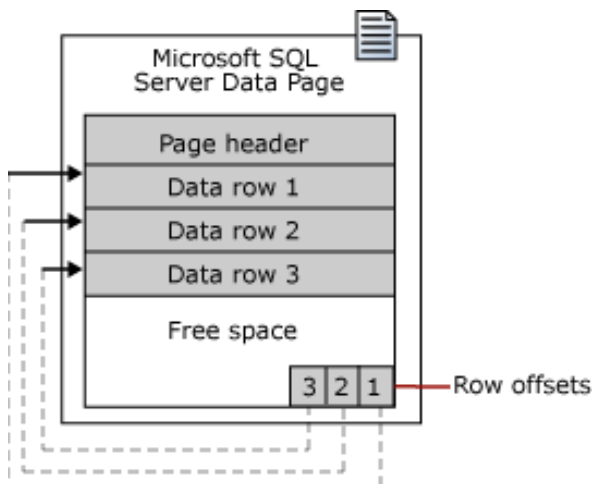
В-дерево: варианты доступа

- индексы на основе В-дерева эффективно поддерживают следующие варианты доступа:
 - поиск уникального ключа;
 - поиск множества ключей (неуникальный индекс, поиск по части ключа, поиск о диапазоне ключей) – результат отсортирован по значению ключа;
 - полный просмотр – результат отсортирован по значению ключа;
- также может быть эффективным быстрый просмотр индекса (просмотр всех страниц в случайном порядке как альтернатива полному сканированию таблицы).



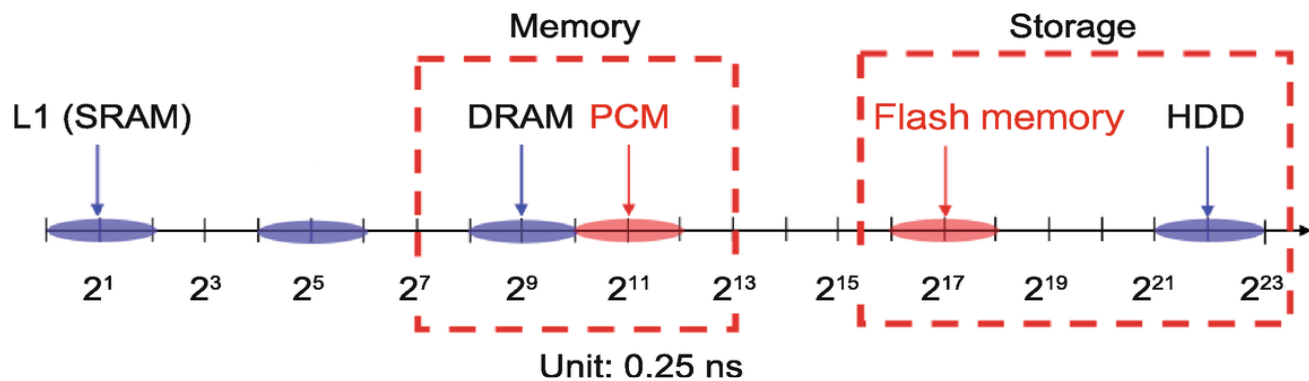
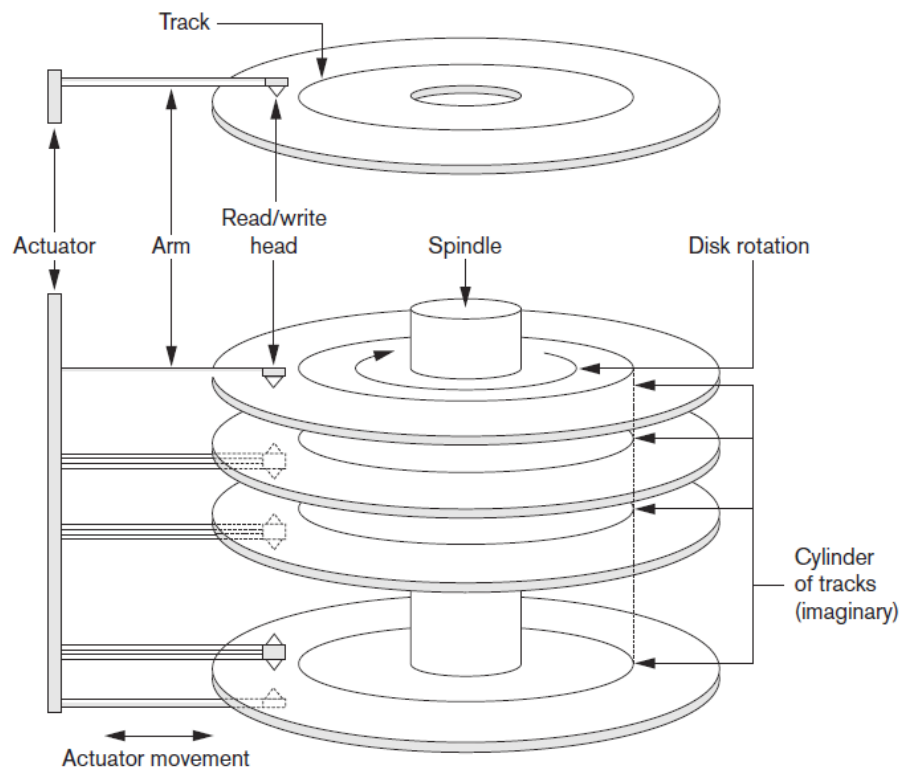
Хранение данных (построчное)

- варианты организации построчного хранения табличных данных в дисковых СУБД:
 - неупорядоченная структура, которая называется кучей (*heap, heap-organized table*);
 - кластеризованная таблица: строки данных хранятся упорядоченно в кластеризованном индексе (индексе, листовые узлы которого хранят всю строку таблицы).



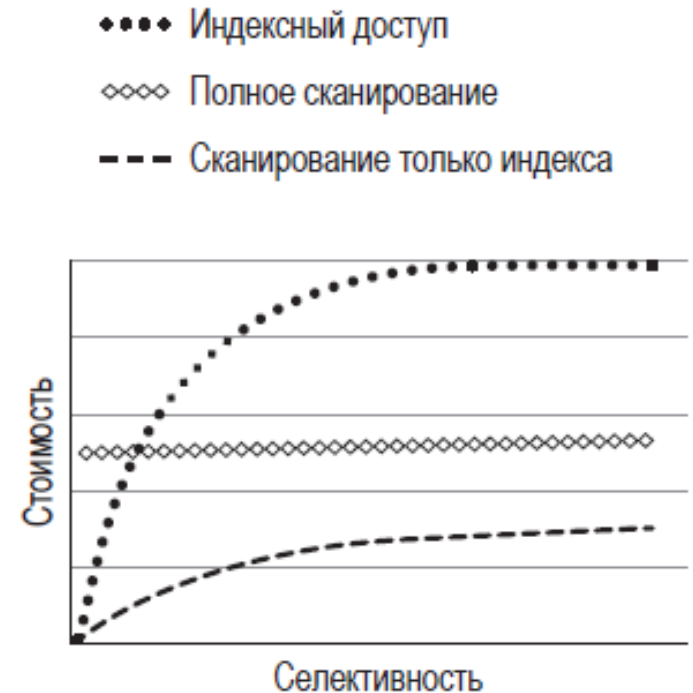
Предпосылки использования индексов

- индексы в СУБД предназначены для:
 - поддержания уникальности значений;
 - ускорения поиска и получения данных по определённым критериям.



Использование индексов

- необходимость использования индексов, их набор и характеристики определяются исходя из набора запросов;
- преимущества: время отклика;
- недостатки:
 - накладные расходы на хранение;
 - накладные расходы на обновление;
 - накладные расходы в процессе использования (конкуренция за ресурсы);
- индексы используются, если:
 - низкая селективность индекса (высокая кардинальность значений в таблице);
 - покрывающий / составной индекс;
- индексы не используются, если:
 - для выполнения запроса нужна большая часть строк таблицы
 - небольшая таблица полностью считывается в оперативную память.



СУБД SQL Server 2025 – Представления, индексы.

Типы представлений

- стандартные: сочетание данных из одной или нескольких таблиц;
- индексируемые: материализованное представление (вычисленное и сохраненное в базе данных), изменения данных в базовых таблицах, отражаются в данных, хранимых в индексируемом представлении;
- секционированные: объединение горизонтально секционированных данных набора базовых таблиц, находящихся на одном или нескольких серверах (локальное / распределенное секционированное представление).

Создание представлений

- определяющий представление запрос может быть инициирован в одной или нескольких базовых таблицах или представлениях;
- столбцу обязательно должно быть присвоено имя, если значение столбца представления формируется выражением и имя не было назначено в самом запросе;
- проверка схемы базовых объектов производится только при обращении к представлению.

```
CREATE VIEW hiredate_view
```

```
AS
```

```
SELECT p.FirstName, p.LastName, e.BusinessEntityID, e.HireDate  
FROM HumanResources.Employee AS e JOIN Person.Person AS p  
      ON e.BusinessEntityID = p.BusinessEntityID;
```

```
CREATE VIEW Sales.SalesPersonPerform
```

```
AS
```

```
SELECT TOP (100) SalesPersonID, SUM(TotalDue) AS TotalSales  
FROM Sales.SalesOrderHeader  
WHERE OrderDate > CONVERT(DATETIME, '20001231', 101)  
GROUP BY SalesPersonID;
```

Создание представлений: привязка к схеме

- при указании SCHEMABINDING базовые таблицы не могут быть изменены каким-либо образом, затрагивающим определение представления – при необходимости таких изменений представление должно быть создано повторно;
- все базовые объекты представления (таблицы, представления и функции) должны располагаться в одной базе данных;
- запрос, определяющий представление, должен использовать полные имена объектов (*схема.объект*).

```
CREATE OR ALTER VIEW Taxi.[Kamaz_cabs]
WITH SCHEMABINDING
AS
SELECT CabID, CarNumber
FROM Taxi.Cab
WHERE Brand = 'Kamaz'
```

Обновляемые представления

- любые изменения, в том числе инструкции UPDATE, INSERT и DELETE, должны ссылаться на столбцы только одной базовой таблицы;
- изменяемые в представлении столбцы должны прямо ссылаться на данные в столбце базовой таблицы - они не могут быть получены другим способом, например:
 - статистическими функциями (AVG, COUNT, SUM, MIN, MAX, GROUPING, STDEV, STDEVP, VAR или VARP);
 - вычислением на основе других столбцов (столбцы, полученные при помощи операторов UNION, UNION ALL, CROSSJOIN, EXCEPT и INTERSECT, также основаны на вычислениях и поэтому недоступны для изменения);
- изменяемые столбцы не могут указываться в предложениях GROUP BY, HAVING и DISTINCT.

Обновление представлений: CHECK OPTION

- при указании CHECK OPTION все изменения данных через представление должны удовлетворять ограничениям запроса, определяющего представление (таким образом, изменения данных не должны влиять на их видимость через представление).

```
CREATE VIEW dbo.SeattleOnly
AS
SELECT p.LastName, p.FirstName, e.JobTitle, a.City,
       sp.StateProvinceCode
FROM HumanResources.Employee e INNER JOIN Person.Person p
    ON p.BusinessEntityID = e.BusinessEntityID
    INNER JOIN Person.BusinessEntityAddress bea
        ON bea.BusinessEntityID = e.BusinessEntityID
    INNER JOIN Person.Address a
        ON a.AddressID = bea.AddressID
    INNER JOIN Person.StateProvince sp
        ON sp.StateProvinceID = a.StateProvinceID
WHERE a.City = 'Seattle'
WITH CHECK OPTION;
```

Изменение и удаление представлений

```
CREATE VIEW HumanResources.EmployeeHireDate
AS
SELECT p.FirstName, p.LastName, e.HireDate
FROM HumanResources.Employee AS e JOIN Person.Person AS p
      ON e.BusinessEntityID = p.BusinessEntityID ;
GO;
```

```
ALTER VIEW HumanResources.EmployeeHireDate
AS
SELECT p.FirstName, p.LastName, e.HireDate
FROM HumanResources.Employee AS e JOIN Person.Person AS p
      ON e.BusinessEntityID = p.BusinessEntityID
WHERE HireDate < CONVERT(DATETIME,'20020101',101) ;
GO
```

```
DROP VIEW IF EXISTS dbo.Reorder;
```


Индексированные представления

- индексировать представление можно, создав для него уникальный кластеризованный индекс:
 - индексированные представления значительно повышают производительность некоторых типов запросов (например, группирующих множество строк);
 - индексированные представления не очень хорошо подходят для часто обновляющихся базовых наборов данных;
- запрос, формирующий индексированное представление:
 - должен ссылаться на единственную таблицу или соединение нескольких таблиц;
 - может выполнять агрегирование данных (GROUP BY) с ограничениями;
 - ключевые столбцы и условия WHERE и GROUP BY могут содержать только точные детерминированные выражения.

Индексированные представления: пример

```
SET NUMERIC_ROUNDABORT OFF;  
SET ANSI_PADDING, ANSI_WARNINGS, CONCAT_NULL_YIELDS_NULL, ARITHABORT, QUOTED_IDENTIFIER,  
ANSI_NULLS ON;
```

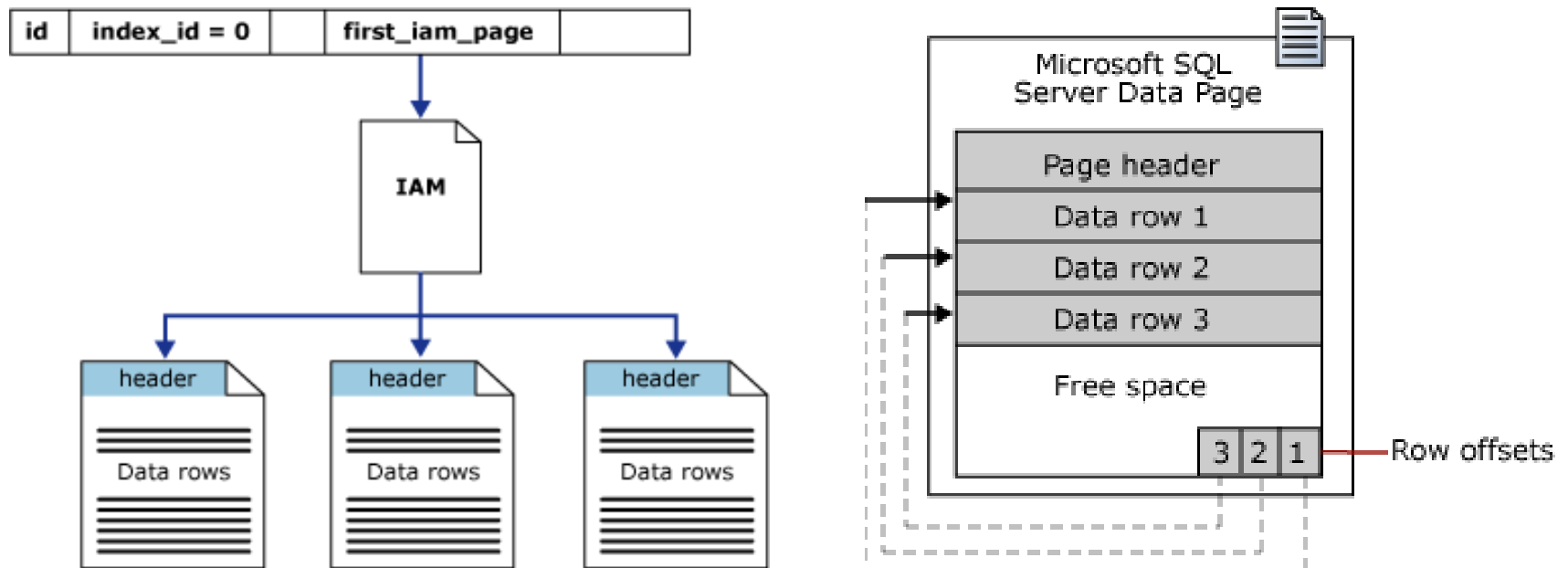
```
CREATE VIEW Sales.vOrders WITH SCHEMABINDING  
AS  
SELECT SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS Revenue,  
       OrderDate, ProductID, COUNT_BIG(*) AS COUNT  
FROM Sales.SalesOrderDetail AS od, Sales.SalesOrderHeader AS o  
WHERE od.SalesOrderID = o.SalesOrderID  
GROUP BY OrderDate, ProductID;  
GO
```

```
CREATE UNIQUE CLUSTERED INDEX IDX_V1 ON Sales.vOrders (  
    OrderDate, ProductID  
);  
GO
```

```
SELECT OrderDate, SUM(UnitPrice * OrderQty * (1.00 - UnitPriceDiscount)) AS Rev  
FROM Sales.SalesOrderDetail AS od INNER JOIN Sales.SalesOrderHeader AS o  
    ON od.SalesOrderID = o.SalesOrderID  
    AND o.OrderDate >= CONVERT(DATETIME, '03/01/2012', 101)  
    AND o.OrderDate < CONVERT(DATETIME, '04/01/2012', 101)  
GROUP BY OrderDate  
ORDER BY OrderDate ASC;
```

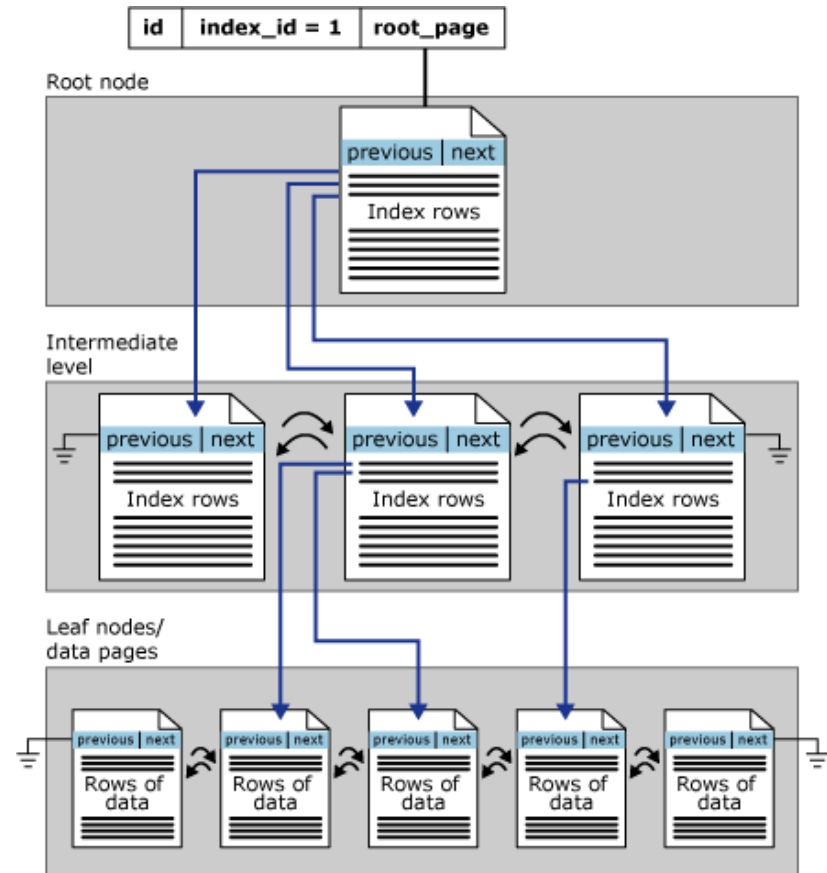
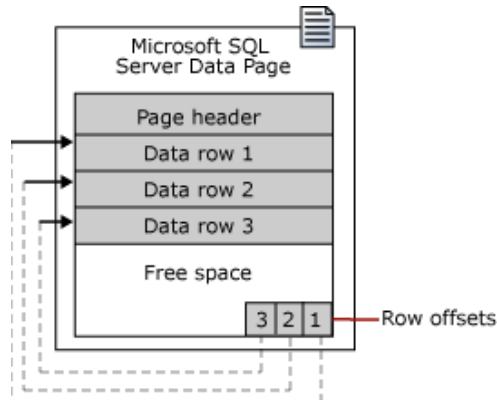
Хранение данных: куча

- если у таблицы нет кластеризованного индекса, то строки данных хранятся в неупорядоченной структуре (куче, *heap*);
- положение строки определяется комбинацией номера страницы и номера строки в странице.



Хранение данных: кластеризованная таблица

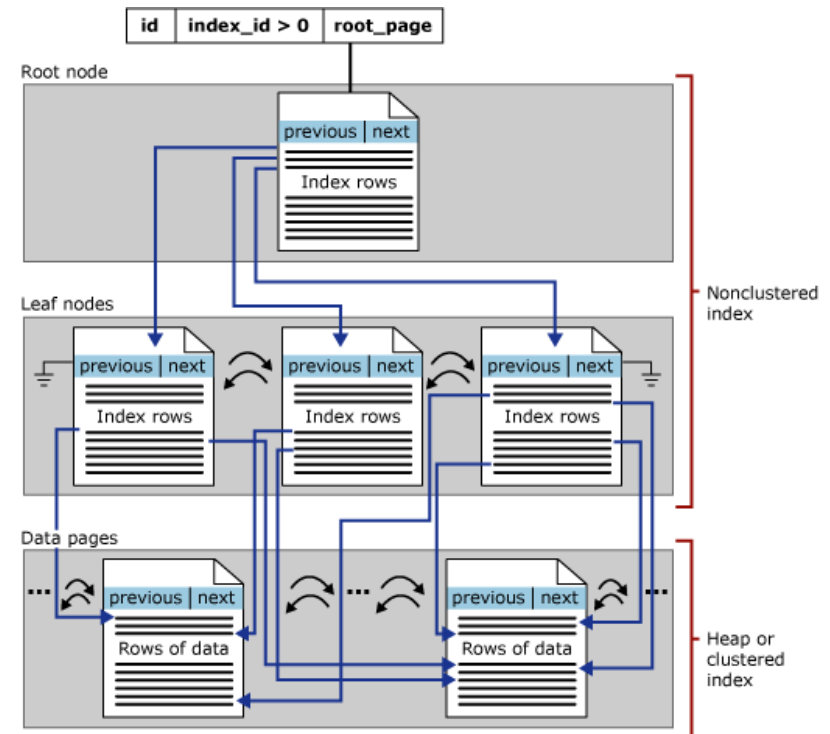
- строки данных хранятся упорядоченно в кластеризованном индексе (*clustered index*):
 - листовые узлы индекса хранят всю строку таблицы (представления), следовательно, строки отсортированы по значению ключа индекса;
 - может быть создан только один кластеризованный индекс для каждой таблицы (представления);
 - положение строки определяется значением ключа.



- ❖ по умолчанию, уникальный кластеризованный индекс создаётся для первичного ключа таблицы

Некластеризованные индексы

- некластеризованные (*non-clustered*) индексы имеют структуру, отдельную от строк данных;
- в некластеризованном индексе содержатся значения его ключа, и каждая запись ключа в листовой странице содержит указатель строки (*row locator*) на соответствующую строку данных:
 - для кучи указатель строки является идентификатором строки (RID, *row identifier*);
 - для кластеризованной таблицы указатель строки является ключом кластеризованного индексе;
- добавление дополнительных неключевых столбцов на конечный уровень некластеризованного индекса позволяет выполнять полностью индексированные (покрывающие, *covering*) запросы.



Проектирование индексов

- при проектировании и создании индексов учитываются следующие аспекты:
 - набор ключевых полей индекса;
 - порядок сортировки индекса;
 - уникальность индекса;
 - добавление неключевых столбцов к конечному уровню некластеризованного индекса;
 - количество, размер и типы столбцов в ключе индекса (до 32 столбцов; общий размер ограничен 900 байтами для кластеризованного индекса и 1700 – для некластеризованного; не допускается использование типов `ntext`, `text`, `varchar(max)`, `nvarchar(max)`, `varbinary(max)`, `xml`, `image`, `json`, `vector`);
 - размещение индексов в файловых группах.

Создание и удаление индексов

- уникальный кластеризованный индекс автоматически создаётся при определении ограничения первичного ключа (PRIMARY KEY);
- уникальный некластеризованный индекс автоматически создаётся при определении ограничения UNIQUE (значение NULL также считается уникальным и может встречаться не более одного раза).

```
CREATE CLUSTERED INDEX IX_TestT_TestC ON dbo.TestTable (TestCol1);
```

```
CREATE NONCLUSTERED INDEX IX_ProductVendor_VendorID  
ON Purchasing.ProductVendor (BusinessEntityID);
```

```
IF EXISTS (SELECT name from sys.indexes  
           WHERE name = N'AK_UnitMeasure_Name'  
           AND object_id = OBJECT_ID(N'Production.UnitMeasure', N'U'))  
DROP INDEX AK_UnitMeasure_Name ON Production.UnitMeasure;  
GO
```

```
CREATE UNIQUE INDEX AK_UnitMeasure_Name ON Production.UnitMeasure (Name);
```

Покрывающие индексы

- покрывающие (*covering*) индексы могут существенно повысить производительность запросов за счёт того, что все столбцы, необходимые для выполнения запроса, присутствуют в индексе (следовательно, не требуются обращения к основной таблице)
 - составные индексы;
 - индексы с дополнительными неключевыми полями.

```
CREATE INDEX IX_VendorID
    ON dbo.ProductVendor (VendorID DESC, Name ASC, Address DESC);
```

```
CREATE NONCLUSTERED INDEX IX_Address_PostalCode
ON Person.Address (PostalCode)
INCLUDE (AddressLine1, AddressLine2, City, StateProvinceID);
GO
```

```
SELECT AddressLine1, AddressLine2, City, StateProvinceID, PostalCode
FROM Person.Address
WHERE PostalCode BETWEEN N'98000' and N'99999';
GO
```


Частичные индексы

- частичные (*filtered*) индексы позволяют проиндексировать только некоторое подмножество строк таблицы (представляющих наибольший интерес с точки зрения бизнес-логики приложения, использующего данные).

```
DROP INDEX IF EXISTS FIBillOfMaterialsWithEndDate  
    ON Production.BillOfMaterials  
GO
```

```
CREATE NONCLUSTERED INDEX FIBillOfMaterialsWithEndDate  
    ON Production.BillOfMaterials (ComponentID, StartDate)  
    WHERE EndDate IS NOT NULL ;  
GO
```

```
SELECT ProductAssemblyID, ComponentID, StartDate  
FROM Production.BillOfMaterials  
WHERE EndDate IS NOT NULL  
    AND ComponentID = 5  
    AND StartDate > '01/01/2008';
```

Изменение и перестроение индексов

```
CREATE INDEX IX_FF ON dbo.FactFinance (FinanceKey ASC, DateKey ASC);
```

```
CREATE INDEX IX_FF ON dbo.FactFinance (FinanceKey, DateKey, OrganizationKey DESC)  
  WITH (DROP_EXISTING = ON, FILLFACTOR = 80);
```

```
-----  
ALTER INDEX PK_Employee_EmployeeID ON HumanResources.Employee REBUILD;
```

```
-----  
ALTER INDEX ALL ON Production.Product REBUILD WITH (FILLFACTOR = 80, SORT_IN_TEMPDB = ON,  
STATISTICS_NORECOMPUTE = ON);
```

```
-----  
ALTER INDEX IX_Employee_ManagerID ON HumanResources.Employee DISABLE;
```

```
-----  
ALTER INDEX PK_Department_DepartmentID ON HumanResources.Department DISABLE;
```

```
ALTER INDEX PK_Department_DepartmentID ON HumanResources.Department REBUILD;
```

```
ALTER TABLE HumanResources.EmployeeDepartmentHistory  
CHECK CONSTRAINT FK_EmployeeDepartmentHistory_Department_DepartmentID;
```

```
-----  
ALTER INDEX test_idx on test_table REBUILD WITH (ONLINE = ON, MAXDOP = 1, RESUMABLE = ON);
```

Вопросы к экзамену

- Представления: назначение, типы. Привязка к схеме. Обновление данных через представления.
- Хранение данных: куча, кластеризованная таблица. Индексы на основе В-дерева. Некластеризованный индекс. Индексированное представление.
- Проектирование и использование индексов. Уникальные, покрывающие и частичные индексы.

Лабораторная работа №7.

Представления и индексы

1. Создать представление на основе одной из таблиц задания 6.
2. Создать представление на основе полей обеих связанных таблиц задания 6.
3. Создать индекс для одной из таблиц задания 6, включив в него дополнительные неключевые поля; привести пример запроса к данным, выполнение которого может быть ускорено при использовании созданного индекса.
4. Создать индексированное представление.